Command Line and Headless Use

What 44helper can do without a browser: its command-line options, its web interface as a scripted API, and the reinstall job that runs unattended under systemd.

--INSTALL | --BOOT-REPORT | HTTP API | UNATTENDED

SUMMARY · READ-ONLY REVIEW

Four ways to use it without the page

WAY	WHAT IT IS FOR	NEEDS
Command-line options	Install, remove, and report on the helper itself	Root
The server, started by systemd	Serves the page and the API, and is restarted if it stops	Nothing from the operator
The HTTP API	A script can log in, start a step, and read the result	The root password
The reinstall job	Reinstalls the suite's tools unattended, then reboots the node	Started from the Config tab, or from the command line by hand

Install steps are described in Stage 1. The tunnels and the terminal are covered in Stages 2 and 5.

The options the program accepts

The program is `44helper.py`, installed at `/usr/local/bin/44helper.py`. With no option it starts the server. The options are parsed by one argument parser; any other argument is an error.

COMMAND	WHAT IT DOES	ROOT
`44helper.py`	Starts the server on the address and port in `/etc/44helper/44helper.conf`. The defaults are 0.0.0.0 and port 9997. Run by systemd in normal use.	Yes, to change the firewall and services
`44helper.py --install`	Installs the helper as a service and starts it (Part 3).	Yes
`44helper.py --uninstall`	Stops and removes the service and its command links (Part 3).	Yes
`44helper.py --boot-report`	Prints the Boot tab's report as text to standard output, then exits.	Not checked in the code
`44helper.py --boot-report --json`	The same report as JSON.	Not checked in the code

The `--boot-report` option redirects the program's own log output to standard error while it builds the report, so the report on standard output is clean enough to save to a file or to parse.

The helper's own service

3.1 What `--install` does

1. Makes sure `/etc/44helper` exists with mode 0700.
2. Copies the running file to `/usr/local/lib/asl_dvs/44helper/` under its own name, and removes the previous version it finds there.
3. Makes symlinks to it at `/usr/local/bin/44helper.py` and at the older path `/opt/44helper/asl_dvs_m17_44helper.py`. A plain file at either path is replaced by the link, and the install says so.
4. Writes the launcher and the service file `/etc/systemd/system/44helper.service`.
5. Opens the helper's port in the firewall (the port in the config, default 9997), using the firewall helper from the shared functions.
6. Reloads systemd, enables `44helper.service`, and restarts it.

The service runs the launcher with the installed file. It is set to Type=notify, to restart always after 3 seconds, and to be checked by a watchdog every 60 seconds. It runs as root, and its output goes to the journal.

3.2 What `--uninstall` does

1. Stops and disables the service.
2. Removes the service file and reloads systemd.
3. Removes the command links at `/usr/local/bin/44helper.py` and the older path, and the installed copies they point to in the install folder.
4. Removes Python cache files and empty folders left behind.

The removal steps in the code work on the links, the installed copies and the folders around them. None of those steps names `/etc/44helper`, so the saved settings are expected to survive. This was read from the code (in part), not from a live node.

Driving the helper from a script

The page talks to the helper over the same JSON API that a script can use. Nothing in the API needs the page itself, but the rules below must be followed.

4.1 Rules for every request

- **Login.** Every route except the public ones needs a session. Without one the helper answers 401 with "Authentication required".
- **The request header.** Every POST needs the header `X-Requested-With: XMLHttpRequest`. Without it the helper answers 403. The header is a guard against forged requests from other sites, not a password.
- **Lockout.** Five failed logins from one address within five minutes lock that address for 15 minutes, and the login answers 429 with the seconds left.
- **Session cookie.** A successful login sets the cookie `helper44_session`. A script keeps that cookie and sends it on later requests.

4.2 The routes a script uses

ROUTE	METHOD	LOGIN	WHAT IT DOES
`/healthz`, `/api/ping`	GET	No	Answers if the helper is running. Useful for a monitor.
`/api/version`	GET	Yes	The helper's version.
`/api/login`	POST	No	Body is JSON with a `password` field, the node's root password. Sets the session cookie.
`/api/logout`	POST	Yes	Ends the session.
`/api/<tab>/script`	GET	Yes	The steps of a tab (for example `/api/asl3/script`), with their exact commands.
`/api/<tab>/action`	POST	Yes	Starts a step. Tabs with this route include asl3, dvswitch, svxlink, m17, amp and cf.
`/api/action_status?job_key=...`	GET	Yes	The state and output of a job.
`/api/config/reinstall`	GET	Yes	The reinstall job's state and the last lines of its log.

A step request carries these fields: `mode` (install, purge or custom), `distro`, `step_id`, `fields` (for steps that need values), and `command_text`, the exact command. The helper runs only the command it builds itself and refuses a different one (Stage 1, Part 3).

4.3 The shape of a scripted install

The sequence below shows the shape of the calls. The placeholders are not real values, and a real password should never be typed into a command line where the shell history keeps it. Read the password from a file with restricted permissions.

```
1. log in             POST /api/login        {"password": "<from a protected file>"}
                        header X-Requested-With: XMLHttpRequest
                        keep the helper44_session cookie
2. read the steps     GET  /api/asl3/script
                        find the step; copy its cmd (or build it from its fields)
3. start the step     POST /api/asl3/action
                        {"mode": "install", "distro": "bookworm",
                         "step_id": "prereqs", "command_text": "<the exact cmd>"}
4. follow the job     GET  /api/action_status?job_key=asl3_install_bookworm_prereqs
                        poll until the status is done; read the output
```

A step whose fields are needed (a callsign, a port) must be sent with `fields` filled in, and `command_text` must be the command the helper would build from them. The simplest reliable way is to copy the command shown by the script route after the fields are set.

Unattended reinstall of the suite's tools

The Config tab can reinstall the suite's tools from a library, then reboot the node. The work is done by a script written to disk and run by systemd, so it keeps going when the page or the helper's session is gone.

5.1 What it reinstalls

TOOL	PORT	CRITICAL
instmon	8990	Yes
Dashboard (nodecontrol)	8989	Yes
SysMon	9999	No
wifimon	8991	No
44helper	9997	No

5.2 Where the new files come from

For each tool that is installed, the job looks in the library folder `/etc/asl_dvs/instmon_library/<tool>/` for the newest version of the same kind. It keeps the Pi Zero 2 W or full-build variant to match what is installed (the version string says "pi02w" for the fork). If the library has no newer copy, it uses the installed copy. A tool with no version line, or no `--install` option, is left alone.

5.3 The order of work

1. **Plan.** Works out which tool gets which file, and writes the plan.
2. **Back up.** Copies each installed file to `config_restore/reinstall_backup`.
3. **instmon first.** Before instmon is replaced, a rollback timer is set for 90 seconds. Then instmon is installed and given 6 seconds. If instmon does not answer on its port within 90 seconds, the timer puts the backup back and reinstalls the old file.
4. **The other tools.** Each is installed with `--install`, then checked: its service must be active and its port must answer, within 60 seconds.
5. **Settle.** Every tool that was reinstalled is checked again.
6. **Reboot or not.** If instmon or the Dashboard is not healthy, or Asterisk is loaded but not active, the job does not reboot and says why. Otherwise it reboots the node 5 seconds later, through a one-shot systemd timer.

After a reboot the job's state changes to "rebooted" once the boot time is later than the scheduled reboot. A reboot that cannot be scheduled leaves the job at "done, no reboot", and the message asks for a reboot by hand.

5.4 Starting it and watching it

- **From the page:** the Config tab shows the plan, then starts the job when the operator confirms. The helper runs the job with systemd-run, under the unit name `asl-dvs-reinstall`, so it is not tied to the helper's process.

- **Plan only, from the command line:** `python3 /etc/asl_dvs/instmon_library/config_restore/reinstall_job.py --plan` prints the plan as JSON and changes nothing.
- **Run from the command line:** `... reinstall_job.py --run` runs the job in the foreground, without the service wrapper. Use `systemd-run` as the helper does, so that the job is not stopped if the terminal closes.
- **Status:** `GET /api/config/reinstall`, or the files in `config_restore`: `reinstall_job.json` (the state), `reinstall.log` (the log), and `reinstall_pending.json` (a job waiting to run).
- **Dismiss:** a finished or pending job can be dismissed. A running job cannot.

What can run without a person

TASK	HOW	STOPS FOR A PERSON?
Install a step that is not manual	The API (Part 4)	No, but a step can fail, and the script must read the result
Install or remove the helper	`--install`, `--uninstall`	No
Reinstall the tools and reboot	The reinstall job (Part 5)	No. It reboots the node if its checks pass.
Health monitoring	`/healthz` with a timer	No
Boot report for a file	`--boot-report --json`	No
Package recovery	The Update tab's steps 9 to 11, through the API	No, but run them in order

6.1 What cannot run without a person

- **Manual steps.** Seven steps are marked manual (Stage 1, Part 7): the ASL3 menu, the Ampersand first run and hub file, the Cloudflare token and dashboard steps, the Cloudflare quick test and the Claude sign-in. The page offers no Run button for them, but the API does not check the flag. A script could send one, and it would be an interactive command with no terminal, so these must be done by hand.
- **Steps that stop on a question.** A package step that asks a question at a prompt stops until someone answers. The install commands use the non-interactive settings (`DEBIAN_FRONTEND=noninteractive`, and the keep-the-old-config options), and the step tables use them throughout (43 occurrences in the file). A step without them can stop at a prompt.

6.2 Gates that protect an unattended run

- The package-health gate refuses a package step while `dpkg --audit` reports broken packages.
- The masked-service gate refuses an install while the tab's services are masked, after trying to clear the masks.
- The exact-command check refuses any command the helper did not build itself. A script that sends a command from an old copy of the script route will be refused, so it must read the route first.
- The lockout stops a script that guesses the root password.

6.3 What to watch for

- Package installs and builds have no timeout. A hung install has to be found by watching the output, not by a limit.
- The reinstall job reboots the node on its own. Schedule it for a quiet time.
- The Custom box in the API runs any command as root (Stage 1, Part 6.3). A script should not use it.
- The root password is the only login. A script that holds it can do everything the page can.

What the command line shows

7.1 One program, several roles

The same file is the server, the installer and the boot reporter. A reader who knows that the helper runs as root from systemd will find the installer and the reporter easy to follow.

7.2 The reinstall job is safer than it looks

It keeps a backup of every file it replaces, puts instmon's backup back if instmon does not answer, and refuses to reboot unless the critical tools are healthy. The book can describe it as a guarded update.

7.3 Scripts need the exact command

The API is not a free shell. A script must read each step from the script route and send that exact command. This makes scripted installs match the page, but it also means a script breaks when the step table changes between releases.

7.4 Not checked

- The exact JSON the script route returns for each tab. The shape was read from the route, not from a live response.
- The launcher file and the body of the uninstall's copy-removal loop, which were read in part.
- The behaviour of the reinstall job on a real node, including the rollback timer.